L M U 🌻 C M S I 3 8 0 2

# LANGUAGES AND AUTOMATA II

HOMEWORK #1 Due: 2026-05-04

## About This Assignment

Though there is only one homework assignment for this course, it is a **term project** and there is an expectation that you begin working on it *no later than the 6th week of class*.

The project is the design and implementation of your own programming language, with the following big-picture requirements:

- ✅ You are to work in a group of 3–6 students.
- ✅ The language you design should be original and of "sufficient complexity."
- ✅ The compiler must be complete and fully functional.
- ✅ You must supply unit tests with 100% test coverage.
- ✅ The project is to be hosted on a public GitHub repository.
- ✅ You must create a companion website for the language.
- ✅ You must give a 5 minute lightning talk presentation on the language on the last day of class.
- ✅ Each group member must contribute meaningfully to the project and its presentation.

- ✅ Each group member must provide a signed affidavit that they have completed all readings and watchings individually.

Detailed requirements are below.

> ***Generative AI is permitted for this project.***
>
> *You are, however, expected to clean up and reorganize any assistant-generated code by removing superfluous comments, renaming its ugly identifier names, restructuring to fit the expected project layout, and everything else required to produce a quality project.*

# Required Readings and Watchings

Although your team will be implementing the language, compiler, companion website, and final presentation together, every team member is **individually responsible** for each of the following:

- ✅ Read Atul Gawande's *The Checklist*.
- ✅ Review all course notes starting from Language Design through Code Optimization.
- ✅ Read the Ohm README, the Ohm Tutorial, and the four pages of documentation (Syntax, API, Philosophy, and Patterns & Pitfalls) linked from the main Ohm Documentation.
- ✅ Read Chapters 1–1.4 of [Mogenson].
- ✅ Skim Ch. 2.1–2.5, 2.10, and 3.1–3.6 of [Mogenson]
- ✅ Read Chapters 4–6 of [Mogenson]
- ✅ Skim Ch. 7 of [Mogenson]
- ✅ Skim Ch. 10–12 of [Mogenson] (paying attention to 11-12)
- ✅ Read this entire overview of Java Language changes from versions 9 through 25. The purpose of this reading is to get you familiar with modern

programming languages features. There is a lot to this, so skim when it gets too detailed.

- ✅ Skim The Wikipedia page on Static Program Analysis
- ✅ Skim the OWASP brief article on Static Code Analysis (from a security perspective)
- ✅ Watch This video (you can play at 1.5x)
- ✅ Browse the Wikipedia article on Program Optimization
- ✅ Study this "Optimization Killers" article. You'll be glad you did. There are so many good things in here you can impress your friends (and future interviewers) with!
- ✅ Browse this great presentation by Graydon Hoare. You don't have to understand every slide, but jot down notes from some of the big ideas. If it gets too technical, don't worry, just move on.
- ✅ Read Frances Allen's classic A Catalogue of Optimizing Transformations. This work followed two of her previous papers where she created new frameworks and algorithms and structures for performing and implementing static program analysis. You may also wish to read this remembrance from IBM (where she was a fellow), or any number of bibliographies on her life and work.

Read these throughout the semester. Don't try to read them all at once.

Keep in mind these materials are not the only readings and watchings required for the course. See the quiz preparation pages for more.

# Detailed Project Requirements

In this class you will not only be writing a compiler—you will be writing a compiler for a language that you *design yourself*. You have wide latitude in designing your language: I want you to design a language you can actually love. The only requirements are: (1) it must be of sufficient complexity (i.e., enough interesting features, including features beyond the example languages we see in class), and (2)

it requires significant work in the static analysis phase. You can satisfy the latter requirement with a statically-typed language with static scoping (in addition to a few other checks of course). It *may* be possible to design a dynamically typed language, provided you define a significant number of statically checked contextual constraints. If you go with a dynamically-typed language, you should check with me first to make sure your workload is fair.

Implement your project according to the following detailed requirements.

## Repository Structure

Your project should end up with a project structured as follows:

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json        -- (Optional)
├── docs
│   └── ...                 -- logo, companion website, etc.
├── examples
│   └── ...                 -- lots of example programs
├── src
│   ├── (yourlanguagename).js
│   ├── (yourlanguagename).ohm
│   ├── compiler.js
│   ├── parser.js
│   ├── core.js
│   ├── analyzer.js
│   ├── optimizer.js
│   └── generator.js
└── test
    ├── compiler.test.js
    ├── parser.test.js
    ├── analyzer.test.js
    ├── optimizer.test.js
    └── generator.test.js
```

Make sure there are no additional, unnecessary files in your repo. A pristine repo is a happy repo! Don't lose points by failing to meet this requirement. Pay attention to your *.gitignore* file!

## Functionality

The command line script to run the compiler, the workings of the compiler itself, and the mechanisms for testing and coverage analysis, should work in the same fashion as the Carlos compiler in the course notes.

## Language Design

Use the Ohm Editor to assist you in designing your language grammar and syntax, using a creating, prototyping style.

Make sure the language is complex enough, especially in its statics. You should have *most* if not all of the following performed at compile time: Scope resolution, proper contextual use of `return`, `break`, etc., type checking, type inference, parameter matching, pattern exhaustiveness, access controls, etc.

If your language is mostly dynamic, make up for the lack of type checking with something really special, like really a really sophisticated concurrency mechanism that will require more sophisticated code generation and optimization in later assignments.

## The README file

Ensure your README has:

- A nice logo (please put some care into this)
- The name of your language, in a large font
- A one paragraph introduction (tell a story!)
- A list of language features

- A description of the required static, safety, and security checks your compiler performs
- Lots of example programs—you can do the "your language on the left, JavaScript on the right" presentation style.

Make sure the README is up-to-date with the actual work you did! If something in your language design did not get implemented, mark the associated content in the README with a "Not yet implemented" warning.

## The package.json file

Include a proper project name and the names of all the team members in the `author` field. Add the property `"type": "module"` to allow for modern JavaScript. Ensure that `npm test` runs tests with coverage. Be complete with all your metadata, including the repository property.

## Tests

100% test coverage! Hold yourself to this goal. Be proud!

## Code Style

Ensure all of the code has been Prettied. Your repo is public and you want to show your best face to the world!

**Add optimizations to your compiler project**. You do not have to do too many, but the few you do should be non-trivial. Because everyone in class is compiling a somewhat different language, you must check with me *early* regarding which optimizations are appropriate for your project. Don't worry, we will negotiate the set of optimizations that you are responsible for, so there should be no surprises when you get your grade.

# Companion Website Requirements

For your language website (a.k.a. a "Home Page") for your programming language using GitHub Pages. The site's page (or pages) should consist of a single page and:

- Be rather pretty, CSS-wise (you can use a template, of course).
- Tell the story of your language in no more than three paragraphs.
- Include examples of programs in your language. You need at least five complete, semantically correct, example programs, that cover every syntactic form of your language, and most if not all of the interesting semantic-level checks.
- Include developer bios (and optionally pictures), because that's kind of fun.
- Include a link to the actual GitHub repo of the compiler.
- Also add a link to this site from the README of the GitHub repo.

# Presentation Requirements

During the last class session, **present, as your entire group, a lightning talk introducing (and showing off!) your language to the rest of the class**. Be personal, be funny, and be brief! You may use slides, video, and even music. If you are using slides, don't use too many, and make sure they are colorful...with images, *and please put these in your project repository*. If your language has a theme (pirates, horror, uwu, vikings, fashion, sci-fi, anime, social policy, medical, whatever), you can dress up (but avoid stereotypes and be respectful).

For tips on how to do a good lightning talk, see here, here, and here. What's a lightning talk? Wikipedia can tell you.

# Checklists

The following checklists are my best effort at helping you make a beautiful and complete project. I might be missing some things, so try to add to it anything you think is important.

## Project

- ✅ Project file structure is beautiful and organized
- ✅ Project GitHub repo has same name as language
- ✅ Project name appears in README
- ✅ Logo appears in README
- ✅ README has story of your language
- ✅ README has the names of all students
- ✅ README lists features of the language
- ✅ README shows examples of programs
- ✅ README contains a link to your GitHub Pages site
- ✅ package.json contains all of the team members in the author property
- ✅ package.json has "type": "module"
- ✅ package.json has lots of metadata
- ✅ Examples folder has at least 5 full example programs and covers the language features
- ✅ .gitignore is present in repo and properly generated (let Github do it for you)
- ✅ No extra stuff in repo (no node_modules, no .DS_Store, etc.)
- ✅ Your project can be cloned
- ✅ I can run npm test immediately after cloning
- ✅ npm test returns all the tests and reports coverage
- ✅ All tests pass
- ✅ Tests have great coverage (you should be able to get 100%, accept nothing less)
- ✅ The Ohm specification is great and complete
- ✅ Your project can be cloned
- ✅ I can syntax check from the command line
- ✅ I can parse from the command line
- ✅ I can analyze from the command line

- ✅ I can optimize from the command line
- ✅ I can generate target code from the command line
- ✅ Errors are detected and reported cleanly when run from the command line
- ✅ The analyzer is well-architected
- ✅ The language is defined with complex static rules
- ✅ Analysis method are written like a pro—they work, and well named, etc.
- ✅ You have at least 50 tests in your suite (5 pts, all or nothing)
- ✅ The list of static constraints in the language is on the README
- ✅ The list of static constraints in the language is on the companion site
- ✅ Analyzer is complete
- ✅ Generator is well structured
- ✅ Generator is complete
- ✅ You have “interesting” code generation
- ✅ You have hundreds of tests
- ✅ The generated code is readable
- ✅ Examples of generated code is on the README or companion site (2 pt)
- ✅ Your language reached the point of being “complex enough.”
- ✅ Code is nice, clean, well-organized, professional
- ✅ Code has been prettied
- ✅ README page is complete, useful, and accurate
- ✅ Companion website is complete, useful, and accurate
- ✅ README page OR companion website has a link to the grammar, making it easy to lookup
- ✅ You have a lot of really good examples in your examples directory
- ✅ Your logo looks good
- ✅ Project commit log shows that you've been working all semester
- ✅ Optional but strongly encouraged: You included something “fun” in the compiler—perhaps an easter egg (document the easter egg in the companion website)

## Companion Website

- ✅ The GH Pages site looks great: nice template, or hand-crafted CSS (3 pts)

- ✅ Site has the story of your language in no more than three paragraphs
- ✅ Site shows at least fives example programs your language
- ✅ Site has developer names and short bios
- ✅ Site points back to the repo

## Presentation

- ✅ You introduced yourselves (each person says their own name)
- ✅ You introduced your language
- ✅ You told the story of your language in a fun and engaging way
- ✅ Slides or other visual aids are compelling
- ✅ If you do have slides (you should) they show some interesting snippets of your language
- ✅ The talk is not too short or too long
- ✅ The talk has some "structure" (prove to me you read the materials I linked to about how to make a lightning talk effective)
- ✅ Intro and outro were compelling, exciting, and memorable!

## Participation

- ✅ As an audience member in presentations, you will be expected to be attentive, respectful, and encouraging, and celebrate and applaud your classmates when they present.
- ✅ As a team member, you are expected to participate actively and contribute fairly to the project. I will be interviewing each team to gauge the contributions of each team member. The old "I participated in meetings but we always committed from so-and-so's computer" isn't really acceptable.

# Submission Instructions

Turn in a single submission for your entire group via BrightSpace, that contains, in text:

- ✅ The names of every student
- ✅ For each student, an affidavit that each of the readings and watchings above were individually completed
- ✅ A link to the public GitHub repository hosting your team's project (which itself should contain a link to the website and a link to the presentation)